ADVANCED AI CAPABILITY PROGRAM FOR PRACTITIONERS

Capstone Projects

Guidelines, Shark Tank Review and Project Catalogue

Prepared for C5i, in partnership with Ambilio

01 ABOUT THE CAPSTONE PROJECTS

The capstone is what makes the ten days applied. The sessions teach how to engineer production software with coding agents; the capstone is where a team takes a real problem from a C5i industry and practice, and builds a working system for it end to end — specified, built, reviewed, evaluated, secured, deployed and defended.

It is deliberately not an exercise. Every brief in this catalogue is shaped like work C5i actually delivers, and the standard of done is the standard the program teaches: a system that runs from a clean clone, carries measured results against a baseline, survives a security review and can be rolled back. Nothing here is disposable at the end of the ten days.

> Every team finishes the program holding a high-value proof of concept it built and can defend — a working system on a real C5i problem shape, engineered and evidenced well enough that C5i can take it further into a client-level solution if it chooses to.
>
> The three highest-scoring projects receive a certificate of appreciation, awarded alongside the program certification.

01.1 EXPECTED OUTCOMES

Every project is judged on what it delivers to two parties — the learner who builds it and C5i, which can take it further. These outcomes apply to every brief in the catalogue, alongside the success metrics stated in each brief.

WHAT IT GIVES THE LEARNER

- Proof of capability, not attendance — a system you built with coding agents and defended in front of practitioners, rather than a course completed.
- The full path once, under supervision — problem to BRD to design to plan to build to review to evaluation to deployment, on one codebase, with the failures happening where they are cheap.
- Judgement about when the agent is wrong — the review habits, the gates and the session discipline that separate an engineer using agents from one being carried by them.
- A portfolio artefact that stands up to questioning, with the evidence pack behind it: review log, evaluation set, security triage, runbook.
- Individual recognition — scoring is per person, so what you personally understood and built is what is on record.

WHAT IT GIVES C5I

- Applied proof that the capability transferred — shipped, measured systems rather than assessment scores alone.
- A portfolio of proofs of concept mapped to C5i's own industries and service practices, each with the evidence needed to judge whether it is worth taking further.
- Any project that earns it can be scaled into a client-level solution, because it was built to a production standard from the start rather than retrofitted afterwards.
- A per-individual read on preparedness and depth across the cohort — who is ready to lead agent-assisted delivery on a client engagement and who needs more ground.
- Reusable engineering assets — instruction files, skills, hooks, evaluation sets and CI gates that outlive the program.

02 CAPSTONE GUIDELINES — COMMON TO EVERY PROJECT

Every team takes one brief from this catalogue, builds it with coding agents to the standard taught across the program, and defends it in a Shark Tank Review Presentation before a practitioner panel. The rules in this section apply to every brief; the catalogues in Sections 03 to 08 are industry-specific.

The brief tells you the business problem, what to build and the numbers you will be judged on. It does not tell you the architecture. Deciding that, specifying it, planning it, driving the agent to build it, reviewing what it produced, and proving the result is safe and operable is the work.

CAPSTONE AT A GLANCE

Catalogue	40 briefs across six industries and ten C5i service practices. Each brief is tagged to one of each.
Project source	One brief per team, chosen from this catalogue. A brief is taken by one team only.
Team size	Teams of three to four participants.
Industries	Banking & Financial Services · Consumer Goods · Retail · Technology · Healthcare & Life Sciences · Manufacturing.
Build window	From Day 3 alongside the sessions, then one week after Day 10 with no sessions running, as dedicated preparation time.
Data	Synthetic or public data only — no real customer data, no PII or PHI, and no C5i or client data of any kind. Generate realistic, real-format test data.
Environment	Claude Code as the primary build environment, the Claude cloud agent for delegated and parallel work, OpenAI Codex where a comparison is useful.
Defence	Shark Tank Review Presentation of Capstone Projects — 20 minutes per team: 15 minutes presented by the team and 5 minutes of panel Q&A. Every team member presents.
Scoring	100-point weighted rubric across six criteria, scored individually for every member of the team.

TEAM AND EXECUTION

- Teams of three to four. Every member owns an area of the build and defends it at the panel.
- One brief per team. Pick the industry you can reason about — the panel will challenge the business case, not only the code.
- Use synthetic or public data only. Generate it in the real format, at a volume that makes the engineering decisions real.
- The build must carry the competencies taught across the ten days — not one day's technique demonstrated ten times.
- Agents do the typing. You own the result: what merges, what ships, and what it does in production is yours to defend.

WHAT “DONE” MEANS

- It runs — a working end-to-end system, demonstrated live from a clean clone, not slideware and not a notebook of fragments.
- It was engineered, not prompted — repository standards, instruction files, skills, hooks, the BRD, the design and the task plan all in version control, with a review record for what the agent produced.
- It is measured — real numbers against the brief's success metrics and the baseline they are compared to, plus an evaluation gate in CI that can block a merge.
- It is secure — scanning clear or explicitly risk-accepted, secrets in a manager and out of agent context, and the agent's permission boundary stated.
- It is operable — deployed to a non-production target with health checks, a demonstrated rollback and an operating runbook.

- It is honest — limitations, residual risk and what the system deliberately refuses to do, stated by the team before the panel has to ask.

PRE-PITCH DELIVERABLES

Submitted before the panel session. Anything not submitted cannot be claimed in the presentation.

- Repository that builds and runs from a clean clone, with a README covering setup, run instructions and an architecture diagram.
- Specification pack — BRD with testable acceptance criteria, solution design, decision records and the sequenced task plan.
- Agent engineering record — instruction files, reusable skills, hooks and CI gates in version control, plus the review log for agent-generated changes and the defects it caught.
- Evaluation report with the metrics named in the brief, the baseline they are measured against, and the evaluation set itself.
- Security evidence — scan output and triage, secrets handling, and the agent access boundary you set.
- Deployment evidence — non-production deployment, health checks, a demonstrated rollback and an operating runbook.
- A 15-minute deck and a one-page summary of architecture, trade-offs and limitations.
- A working demo — live, with a recorded fallback in case of network failure.

02.1 SHARK TANK REVIEW PRESENTATION OF CAPSTONE PROJECTS — 20 MINUTES

Fifteen minutes presented by the team and five minutes of panel Q&A, one week after Day 10. Present the working system and walk its design, the way the agent was driven and controlled, the review and evaluation evidence, the security posture and the deployment. The panel challenges the team live and the team defends its decisions. Every team member presents, and defends their own area under questioning. A system that does not run live cannot pass the demonstration criterion.

FIFTEEN-MINUTE PRESENTATION — SUGGESTED SPLIT

TIME	SEGMENT
2 min	Problem and business case — the pain, who carries it, and the cost of the status quo.
2 min	Solution design — the BRD, the decisions you took and rejected, and the task plan the agent executed.
2 min	How the agent was used and controlled — standards, context setup, delegation, session discipline, and what you refused to merge.
5 min	Live demonstration — the working system on real-format synthetic inputs.
3 min	Evidence — code review, evaluation against the baseline, security posture, deployment and rollback.
1 min	Limitations and next steps — what it does not do yet, and what you would do next.
5 min	Panel Q&A — the panel tests architecture, trade-offs, evaluation, cost and security. Every member answers on their own area.

EVALUATION RUBRIC

CRITERION	WEIGHT	WHAT IS ASSESSED
Functionality	25%	The system works end to end, runs live from a clean clone, and does what the brief asked.
Engineering and delivery discipline	20%	Standards, context and project setup; specification and planning; how the agent was driven and what was allowed to merge; security, deployment and a demonstrated rollback.

CRITERION	WEIGHT	WHAT IS ASSESSED
Evaluation and evidence	20%	Measured results against a stated baseline, a working evaluation gate in CI, and a review record showing defects were caught.
Relevance to C5i	10%	How closely the build maps to work C5i actually delivers, and whether the business case holds up under challenge.
Subject matter understanding	15%	Command of the techniques used and the problem domain — why this approach, what the alternatives were, and where the method breaks. Tested individually under questioning.
Presentation quality	10%	Clarity of the pitch and the deck, and the quality of answers under Q&A.

> Scoring is individual, not by team. The project is built and presented as a group, but every criterion is scored separately for each member of the group, on the area they own and on what they can defend under questioning.
>
> A strong system carried by one or two members will not earn the same score for the rest. Split the build, split the presentation, and make sure every member can answer for the whole design — not only the part they wrote.

PRESENTATION DECK

Five to six slides, no more. The deck supports the live demonstration; it does not replace it.

- Problem statement — the brief, who carries the pain, and the cost of the status quo.
- Solution — what you built and the design decisions behind it, including the ones you rejected.
- Methodology — how the work was specified, planned and driven with the agent, and how the output was reviewed.
- Experiment and outcomes — what you tried, what did not work, and what changed as a result.
- Results and evaluation — the metrics from the brief against the baseline, plus security and deployment evidence.
- Conclusion and future scope — limitations, residual risk, and what would take this from a proof of concept to production.

02.2 COVERAGE ACROSS C5I PRACTICES

Every brief is tagged to one C5i service practice and one industry. The catalogue is built so that a cohort working across it exercises the full practice portfolio rather than clustering on one.

SERVICE PRACTICE	PROJECT NUMBERS
AI & Data Strategy / Consulting	08, 27, 40
Generative AI & Agentic AI	02, 07, 09, 17, 26, 29, 36
AI/ML Solutions & Applications	05, 11, 30, 35
Data Engineering & Data Modernization	03, 10, 18, 31, 37
Analytics & Decision Intelligence	01, 32, 39
Customer & Digital Analytics	06, 16, 25
Marketing & Sales Analytics	13, 20, 22
Supply Chain & Operations Analytics	14, 19, 23, 34
Pricing & Revenue Optimization	15, 24, 33

SERVICE PRACTICE	PROJECT NUMBERS
AI-powered Automation & Workflows	04, 12, 21, 28, 38

INDUSTRY	PROJECT NUMBERS
Banking & Financial Services	8 projects — 01, 02, 03, 04, 05, 06, 07, 08
Healthcare & Life Sciences	6 projects — 09, 10, 11, 12, 13, 14
Retail	7 projects — 15, 16, 17, 18, 19, 20, 21
Consumer Goods	6 projects — 22, 23, 24, 25, 26, 27
Technology	6 projects — 28, 29, 30, 31, 32, 33
Manufacturing	7 projects — 34, 35, 36, 37, 38, 39, 40

02.3 HOW TO SELECT YOUR PROJECT

Teams are formed and projects selected before the build begins. Read the catalogue in Sections 03 to 08 in full before you choose.

FORMING YOUR TEAM

- **Team size.** Work in a team of three to four members.
- **Your choice.** You decide your own team members.
- **Shared domain.** Form the team with colleagues who work in the same or a closely related domain at C5i, so every member can reason about the business problem from their own work.

CHOOSING YOUR PROJECT

- **Match your domain of work.** Select the brief that matches the industry and service practice you currently work in at C5i. The panel challenges the business case, not only the code, and domain fit is what makes the build relevant to your client work.
- **One brief per team.** A brief is taken by one team only.
- **Know the outcome you are signing up for.** Read the brief's success metrics and the expected outcomes in Section 01.1 before selecting — these are what your project is judged on.

SUBMITTING YOUR SELECTION

You will be given a selection form. One submission per team, with the following details:

- Your name.
- Project title, as listed in this catalogue.
- Names and email IDs of all team members.

> Project selection by domain is a key expectation from C5i's stakeholders. Choose the brief closest to the work you do at C5i — that is what makes the project relevant, the learning transferable, and the result worth taking further.

03 BANKING & FINANCIAL SERVICES

8 capstone briefs. Briefs for regulated decisioning, servicing and reporting systems, where an unexplained decision is a compliance problem and not only a quality one. Each is scoped for a team of three to four from Day 3 to the panel, on synthetic data, and states the business problem, what to build, and the success metrics it will be judged on.

01 BANKING & FINANCIAL SERVICES · ANALYTICS & DECISION INTELLIGENCE

Collections Prioritisation Workbench

Business problem	Collections teams work queues ordered by days-past-due and outstanding balance, which sorts by size rather than by what will actually be recovered. Treatment choice — call, message, restructure offer — is made by habit, and nobody can trace why one account was worked before another. Any change to the ordering has to be defensible to a supervisor and to audit, which is why the ordering never changes.
What to build	Write the BRD with testable acceptance criteria into the repository before any code, then build against the standards the agent reads on every run. Score synthetic delinquency accounts on recovery propensity, expose the ranking through a versioned API and an operations interface that shows the drivers behind each account's position, and let a supervisor override with a recorded reason. Put an evaluation gate in CI that blocks a merge when ranking quality regresses against a held-out period. Deploy with a canary and a demonstrated rollback.
Success metrics	Recovered value per worked account against a days-past-due baseline, ranking stability across runs, override rate and reason coverage, p95 API latency, blocked-merge demonstration.
Key techniques	• BRD and acceptance criteria as version-controlled repository artefacts • Scoring service behind a versioned API • Driver-level explanation surfaced in the operations interface • CI evaluation gate on ranking regression • Canary release with automated rollback

02 BANKING & FINANCIAL SERVICES · GENERATIVE AI & AGENTIC AI

Relationship Manager Briefing Agent

Business problem	Relationship managers walk into client conversations having skimmed whatever they could find across CRM notes, holdings and product updates. Preparation quality varies by who prepared it, and the wrong product mentioned to the wrong client is a suitability problem, not a missed sale. The assembly work is mechanical; the judgement is concentrated in two or three decisions.
What to build	Build an agent that assembles a pre-meeting brief from mock CRM, holdings and product systems exposed as MCP tools with typed schemas and scoped credentials. Design the run loop explicitly — iteration limit, stopping condition, budget ceiling — and escalate to a human when the brief would include a product outside the client's stated suitability band rather than dropping it silently. Render the brief in a production interface with every claim linked to the record it came from. Log each tool call with caller, arguments and result at the tool boundary.
Success metrics	Brief quality against a rubric, tool-call success rate, escalation correctness, cost per brief, budget-ceiling adherence.
Key techniques	• MCP tool design with typed schemas and scoped credentials • Run control — iteration limits, stopping conditions, budget ceilings • Source-linked rendering of every generated claim • Suitability-threshold escalation to a human • Tool-boundary audit logging

03 BANKING & FINANCIAL SERVICES · DATA ENGINEERING & DATA MODERNIZATION

Legacy Extract Modernisation with a Code Graph

Business problem	The nightly extracts feeding regulatory and management reporting are a decade of accumulated procedures and scripts that no current employee wrote. Nobody can say which fields are still consumed or what a change would break, so the extracts are never touched and every new requirement is bolted on alongside the old ones. This is the situation most real engagements start from.
What to build	Take the supplied legacy extract codebase you did not write. Generate a code graph over it, produce an architecture map, a dependency and impact analysis, and reconstruct the business rules and field lineage from code alone — while recognising where the agent will confidently mislead you. Write characterisation tests that pin current behaviour, then have the agent re-implement one extract path on a modern pipeline, held inside the intended blast radius. Prove output equivalence against the legacy path on a synthetic dataset before anything is switched.
Success metrics	Rule-reconstruction accuracy against the supplied key, characterisation-test coverage of the migrated path, output equivalence rate against legacy, impact-analysis completeness, migrated-path runtime.
Key techniques	• Code-graph generation and navigation over an unfamiliar repository • Business-rule and lineage reconstruction from code alone • Characterisation testing before any refactor • Blast-radius control on agent-driven change • Output-equivalence verification against the legacy path

04 BANKING & FINANCIAL SERVICES · AI-POWERED AUTOMATION & WORKFLOWS

Trade Finance Document Check Workflow

Business problem	Documentary credit checking is a rules-heavy read of shipping documents against terms, done by hand under a deadline. A discrepancy missed is a payment risk; a discrepancy invented is a delayed transaction and an irritated client. The volume is steady and the rules are written down, which is exactly the shape that gets automated badly.
What to build	Build a workflow service that ingests synthetic document sets, extracts the checkable fields under a fixed schema, evaluates them against the credit terms and produces a discrepancy report citing the clause behind each finding. Handle the unreadable document and the unmappable term by declaring the gap rather than passing the check. Route anything above a value threshold or below a confidence threshold to a human. Put the extraction and rule-evaluation sets in CI as a merge gate and show a blocked merge.
Success metrics	Discrepancy precision and recall on a labelled set, false-discrepancy rate, gap-declaration correctness, straight-through rate at the chosen thresholds, blocked-merge evidence.
Key techniques	• Schema-constrained extraction from mixed-format documents • Rule evaluation with explicit gap and conflict handling • Confidence and value thresholds routing to human review • Labelled-set precision and recall measurement • CI merge gates on extraction and rule quality

05 BANKING & FINANCIAL SERVICES · AI/ML SOLUTIONS & APPLICATIONS

Fraud Alert Triage Console

Business problem	Analysts work model-generated alerts through spreadsheets with no view of why an alert scored the way it did and no visibility of how the model is behaving over time. When the fraud pattern shifts, the first signal is a rising false-positive rate that nobody attributes to drift for weeks.
What to build	Serve the supplied scoring model behind an API and build the analyst console around it — alert queue, feature-level explanation per alert, disposition capture. Instrument latency, throughput and score distribution against stated SLOs, and wire drift detection over the scoring inputs with alerting thresholds. Load-test to the stated throughput target and show the SLO holding. Demonstrate a drift alert firing on an injected distribution shift.
Success metrics	p95 latency at target throughput, SLO adherence under load, drift-alert lead time on the injected shift, disposition-capture completeness.
Key techniques	• Model serving behind a versioned API • Feature-level explanation surfacing in an operations interface • SLO instrumentation and load testing to a stated target • Input-distribution drift detection with alert thresholds • Fault injection to validate detection coverage

06 BANKING & FINANCIAL SERVICES · CUSTOMER & DIGITAL ANALYTICS

Digital Onboarding Drop-off Diagnostic

Business problem	A meaningful share of digital account openings are abandoned mid-journey, and the product team argues about why from aggregate funnel charts. The evidence that would settle it — event streams, device context, error logs, support contacts — sits in different systems and is never joined for a single session.
What to build	Build a diagnostic application that reconstructs individual synthetic onboarding sessions end to end, joins event, error and support signals, and groups abandonment into causes with an example session behind each. Ship it as a real interface with loading, empty and error states, not a notebook. Hold the interface to a design system and run visual-regression and accessibility checks as gates. Report where the diagnosis is supported by evidence and where the data cannot carry a conclusion.
Success metrics	Cause-assignment accuracy against a labelled session set, share of sessions with a supported diagnosis, p95 page load, visual-regression and accessibility gate pass.
Key techniques	• Session reconstruction across multiple event sources • Cause grouping with example evidence attached • Prototype-to-production interface against a design system • Visual-regression and accessibility checks as quality gates • Explicit scoping of unsupported conclusions

07 BANKING & FINANCIAL SERVICES · GENERATIVE AI & AGENTIC AI

Regulatory Change Impact Assistant

Business problem	When a regulation changes, someone has to establish which internal policies, controls and processes it touches. That question is inherently multi-hop — regulation to obligation to control to process owner — and single-shot search over a document store answers it confidently on one hop of evidence. The failure is invisible because the answer reads well.
What to build	Build retrieval over a synthetic corpus of regulations, internal policies and a control library, combining a graph of obligation-to-control relationships with vector retrieval over the text, and route each query to the path that answers it better. Decompose compound questions, retrieve per hop, and attribute citations per hop with the decomposition trace visible. Trigger a corrective pass on low-confidence retrieval instead of generating anyway. Evaluate on a golden set of genuinely multi-hop questions plus single-hop controls, reporting each separately.
Success metrics	Multi-hop accuracy against a single-shot baseline, single-hop control accuracy with no regression, per-hop citation correctness, refusal correctness on uncovered changes, cost per query by path.
Key techniques	· GraphRAG over an obligation-and-control graph · Query decomposition with per-hop citation attribution · Graph-versus-vector routing per query class · Corrective pass on low-confidence retrieval · Multi-hop and single-hop golden-set construction

08 BANKING & FINANCIAL SERVICES · AI & DATA STRATEGY / CONSULTING

Model and Agent Inventory Register

Business problem	Nobody can produce a current list of the models and agents running in the business — who owns them, what data they touch, when they were last evaluated. The register that exists is a spreadsheet someone updates before an audit. The gap is not policy; it is that no system makes registration the path of least resistance.
What to build	Build the register as working software: registration workflow, ownership and data-classification fields, evaluation and review dates, and an API that deployment pipelines call to create or update an entry automatically. Enforce the standard mechanically — a deployment without a register entry fails the pipeline. Model the access rules so sensitive entries are visible only to owners and governance, and test them. Demonstrate the full path from a pipeline run to a register entry and an audit export.
Success metrics	Registration coverage achieved through the automated path, pipeline-gate enforcement demonstration, access-control correctness against the stated rules, audit-export completeness.
Key techniques	· Requirements and governance model written before the build · Mechanical enforcement through pipeline gates · Role-based access design with tested rules · API-first service with an operations interface · Auditable change history and export

04 HEALTHCARE & LIFE SCIENCES

6 capstone briefs. Briefs where a confidently wrong answer is a safety or compliance event, and what the system refuses is judged as closely as what it produces. Each is scoped for a team of three to four from Day 3 to the panel, on synthetic data, and states the business problem, what to build, and the success metrics it will be judged on.

09 HEALTHCARE & LIFE SCIENCES · GENERATIVE AI & AGENTIC AI

Medical Information Response Assistant

Business problem	Medical information teams answer unsolicited product questions under strict constraints — answers must come from approved sources, stay on-label, and stop where the evidence stops. A plausible answer assembled from adjacent material is a compliance event, so the system has to be judged on what it declines as much as on what it answers.
What to build	Build retrieval over a synthetic approved-response and label corpus with citations to the specific document and version. Calibrate confidence and implement an explicit refusal that returns an escalation path rather than a constructed answer. Curate a golden set containing off-label, adversarial and out-of-scope questions, and measure refusal correctness alongside faithfulness. Record every answered and refused query for review, and document the failure modes you could not eliminate.
Success metrics	Faithfulness, citation-grounding rate, refusal correctness on out-of-scope and off-label questions, fabricated-answer rate, p95 latency.
Key techniques	· Version-aware grounded retrieval with document-level citation · Confidence calibration and calibrated refusal · Adversarial and out-of-scope golden-set evaluation · Escalation routing on refusal · Query and answer audit logging

10 HEALTHCARE & LIFE SCIENCES · DATA ENGINEERING & DATA MODERNIZATION

Study Data Ingestion and Quality Gate

Business problem	Study data arrives from sites and vendors in formats that differ by source and change without notice. Downstream analysis runs on whatever came through, and a silently mis-mapped field is discovered late, when re-running the analysis is expensive and the conclusion has already been circulated.
What to build	Build the ingestion pipeline for synthetic study data with schema, range, completeness and cardinality checks applied as gates rather than reports — a failing batch does not land. Detect format and distribution drift on incoming files and raise it as an alert with lineage back to the source feed. Generate the per-source mapping code from the specification with the agent and hold it to the repository standard: documented, typed and tested before it can merge. Inject deliberate degradations and report which checks catch them and how quickly.
Success metrics	Check coverage across the declared schema, detection precision and recall on injected degradations, detection lag, lineage completeness, batch throughput.
Key techniques	· Data-quality checks as landing gates, not reports · Format and distribution drift detection on incoming files · Fault injection to validate detection coverage · Lineage from alert back to source feed · Agent-generated mapping code under an enforced review standard

11 HEALTHCARE & LIFE SCIENCES · AI/ML SOLUTIONS & APPLICATIONS

Appointment No-Show Intervention Service

Business problem	Missed appointments waste clinical capacity that was already scarce, and the reminder process treats every booking identically. Targeting the intervention needs a risk estimate per appointment, served fast enough to act on at booking time. The model exists; operations cannot consume it because it lives in a notebook.
What to build	Package the supplied risk model behind an API with liveness and readiness probes and an autoscaling policy, and build the booking-side integration that calls it. Add a policy layer that converts a score into an intervention decision under stated capacity constraints, so the service recommends an action rather than emitting a number. Build CI with evaluation gates and a tested rollback, trigger the rollback deliberately, and report cold-start behaviour under a load profile matching clinic hours.
Success metrics	p95 latency, cold-start time, SLO adherence under the modelled load, rollback success, intervention-policy adherence to stated capacity limits.
Key techniques	• Container packaging and managed-runtime deployment • Liveness and readiness probes with autoscaling policy • Policy layer converting scores into decisions under constraints • CI evaluation gates with a tested rollback path • Load profiling and cold-start measurement

12 HEALTHCARE & LIFE SCIENCES · AI-POWERED AUTOMATION & WORKFLOWS

Adverse Event Intake Agent

Business problem	Safety case intake arrives as free text from several channels and is keyed by hand into a structured case against a regulated clock. The work is repetitive, the deadlines are legal, and both a missed case and a wrongly created duplicate are reportable problems. A workflow that assumes every system call succeeds will create duplicates in its first week.
What to build	Build a multi-step agent that extracts case fields under a fixed schema, checks minimum-criteria completeness, drafts the narrative and creates the case through an MCP tool against a mock safety system. Engineer for partial failure: idempotency keys so a retry cannot duplicate a case, timeouts with defined fallback behaviour, and checkpointing so a run resumes rather than restarts. Gate case creation behind human confirmation. Run an injected-outage test and report which guarantees held.
Success metrics	Field-extraction accuracy, minimum-criteria detection recall, duplicate-case rate under forced retries, recovery-from-checkpoint rate, human-confirmation correctness.
Key techniques	• Schema-constrained extraction with completeness checking • MCP tool integration with typed interfaces • Idempotency keys on write operations • Checkpointed recovery and injected-outage testing • Human confirmation before any commitment step

13 HEALTHCARE & LIFE SCIENCES · MARKETING & SALES ANALYTICS

Field Engagement Next-Best-Action Service

Business problem	Field teams plan the week from a static call list and their own memory of what happened last time. Engagement history, content response and territory coverage gaps all exist, and none of them combine into a recommendation anyone can act on between two appointments.
What to build	Specify and build a service that produces a ranked next-best-action per contact with the reason attached, and a field-facing interface that works on a phone. Hold the interface to a design system and make loading, offline and error states real rather than placeholders. Put an evaluation gate in CI comparing recommendation quality against a held-out period, and expose a feedback path so a rejected recommendation is captured with its reason and lands in the evaluation set.
Success metrics	Recommendation acceptance against a static-list baseline, reason coverage, p95 load time on a mobile profile, feedback capture rate, gate-blocked regression demonstration.
Key techniques	• Recommendation service with reason attachment • Responsive production interface with real state handling • Feedback capture wired back into the evaluation set • CI evaluation gates on recommendation quality • Design-system and accessibility conformance

14 HEALTHCARE & LIFE SCIENCES · SUPPLY CHAIN & OPERATIONS ANALYTICS

Cold Chain Excursion Investigation Application

Business problem	A temperature excursion in transit triggers an investigation that pulls sensor traces, shipment records, carrier terms and standard procedures together by hand while the product sits quarantined. The delay is the cost, and the conclusion has to be defensible months later.
What to build	Build an investigation application that assembles the excursion timeline from synthetic sensor and shipment data, retrieves the applicable procedure and carrier-term clauses with citations, and produces a draft disposition with the evidence behind it. Degrade honestly when a source is missing — state the gap rather than inferring the missing leg. Build an evaluation set of excursion scenarios and measure disposition accuracy and time-to-conclusion against a manual baseline.
Success metrics	Disposition accuracy on the scenario set, citation correctness, gap-declaration correctness under missing data, time-to-conclusion against a manual baseline.
Key techniques	• Timeline assembly across sensor and transactional sources • Grounded retrieval over procedures and contract terms • Graceful degradation under missing inputs • Scenario-based accuracy evaluation • Evidence-linked draft output

05 RETAIL

7 capstone briefs. Briefs for high-volume decisioning and store-facing systems, where latency, consistency and a defensible method matter more than model sophistication. Each is scoped for a team of three to four from Day 3 to the panel, on synthetic data, and states the business problem, what to build, and the success metrics it will be judged on.

15 RETAIL · PRICING & REVENUE OPTIMIZATION

Markdown Decision Console

Business problem	Markdown decisions are taken late, in spreadsheets, against a calendar rather than against sell-through. Going early costs margin, going late costs inventory, and neither is visible until the season closes and the argument is retrospective.
What to build	Build a console that recommends markdown depth and timing per product-location group from synthetic sell-through and inventory data, with the constraint set — price floor, brand rules, stock cover — held as version-controlled configuration rather than buried in code. Show the projected outcome and the trade-off against the alternative depths considered, including the ones that lose. Let a merchandiser override with a recorded reason and capture the outcome so recommendations can be scored later. Gate merges on a backtest against a held-out season.
Success metrics	Margin and sell-through against a calendar-markdown baseline on backtest, constraint-violation rate with a target of zero, override rate and reason coverage, backtest gate demonstration.
Key techniques	• Constraint set as version-controlled configuration • Scenario comparison surfaced in the decision interface • Override capture with outcome tracking • Backtest as a CI merge gate • Decision-service API design

16 RETAIL · CUSTOMER & DIGITAL ANALYTICS

On-Site Search Relevance Service

Business problem	A large share of site searches return results the shopper does not click, and the team cannot tell whether the problem is query understanding, the index or the ranking. Changes ship on intuition and are judged on next week's conversion number, which moves for a dozen other reasons.
What to build	Build a retrieval service over a synthetic catalogue combining keyword and semantic matching with a reranking stage and attribute filters applied before retrieval. Build a labelled relevance set and measure each stage's contribution by ablation, so the team can say what the reranker is actually worth. Put the relevance evaluation in CI as a gate. Expose the service behind an API with caching, and report cost and latency per query at each configuration.
Success metrics	Precision at k and ranking quality against the labelled set, per-stage ablation deltas, cache hit rate, p95 latency and cost per query, gate-blocked regression.
Key techniques	• Hybrid retrieval with reranking and pre-retrieval filters • Labelled relevance-set construction • Ablation testing of each pipeline stage • Caching with cost and latency measurement • Relevance gates in CI

17 RETAIL · GENERATIVE AI & AGENTIC AI

Store Operations Knowledge Assistant

Business problem	Store staff resolve situations against procedures, promotion terms and safety rules spread across dozens of documents updated on different cycles. Asking a supervisor is faster than searching, so the documented answer loses to the remembered one — and the remembered one is often superseded.
What to build	Build retrieval over synthetic procedures, promotion terms and safety rules carrying effective-date metadata, routing a query to the right sub-corpus before retrieval. Cite the specific clause and its effective date, and refuse when retrieval quality falls below threshold rather than guessing. Build the interface for a store device — fast first response, legible, explicit about confidence. Evaluate on a set that deliberately includes superseded and unanswerable questions.
Success metrics	Answer accuracy on the curated set, refusal correctness on superseded and unanswerable questions, effective-date citation accuracy, time to first token, p95 latency.
Key techniques	• Intent routing across sub-corpora before retrieval • Effective-date-aware grounded citation • Confidence calibration and explicit refusal • Streaming interface with time-to-first-token targets • Golden set including superseded content

18 RETAIL · DATA ENGINEERING & DATA MODERNIZATION

Product Master Consolidation Service

Business problem	The same product exists several times across systems under different codes, descriptions and supplier names, fragmenting stock, sales and margin views. Fuzzy matching misses real variants and merges records that should stay apart, and because new items arrive continuously a one-off cleanup does not hold.
What to build	Build the matching service as a running pipeline rather than a cleanup script: blocking to reduce the candidate space, similarity matching with a threshold tuned on a labelled pair set, and a review queue for the band where the system should not decide alone. Operate it incrementally with high-water-mark tracking, a stated freshness target and lineage to source. Compare against a fuzzy-match baseline and defend the threshold you chose against the review capacity you assumed.
Success metrics	Pair-level precision and recall against a fuzzy-match baseline, review-queue volume at the chosen threshold, re-processing lag against the freshness target, lineage completeness.
Key techniques	• Blocking and similarity-based entity matching • Threshold tuning against a labelled pair set • Human review band design sized to capacity • Incremental processing with high-water-mark tracking • Lineage to source and stated freshness targets

19 RETAIL · SUPPLY CHAIN & OPERATIONS ANALYTICS

Replenishment Exception Console

Business problem	Replenishment runs automatically and the exceptions — a store that will stock out before the next delivery, an order the system suppressed — are worked from a flat report that grows faster than anyone can read it. Staff triage by store familiarity rather than by consequence.
What to build	Build a service that classifies replenishment exceptions by cause and expected impact, and an operations console that orders them by what they cost if ignored. Show the evidence behind each classification and let the user act — approve, suppress, escalate — with the action recorded. Instrument the service against stated latency and throughput SLOs and load-test to the daily peak. Evaluate classification accuracy on a labelled exception set and ordering quality against realised impact.
Success metrics	Classification accuracy on the labelled set, ordering quality against realised impact, p95 latency at peak load, action-capture completeness, SLO adherence.
Key techniques	• Exception classification with evidence surfacing • Impact-ordered queue design • Action capture and audit trail • SLO instrumentation and peak load testing • Labelled-set accuracy measurement

20 RETAIL · MARKETING & SALES ANALYTICS

Promotion Performance Attribution Application

Business problem	Promotion post-mortems are argued rather than measured. Uplift is quoted against a baseline nobody agrees on, and because the calculation lives in a personal spreadsheet the same promotion can be a success or a failure depending on who reports it.
What to build	Build the attribution as a service with the baseline definition, control selection and uplift calculation written into code, documented and version-controlled — the method is the deliverable as much as the number. Expose results through an interface that shows the counterfactual and the confidence interval, not just the headline. Validate against synthetic data carrying a planted effect and show the service recovers it within a stated tolerance. Gate merges on that recovery test so a method change cannot ship unmeasured.
Success metrics	Recovery of the planted effect within a stated tolerance, baseline-definition test coverage, reproducibility of a result across runs, gate demonstration on a method change.
Key techniques	• Method as documented, version-controlled code • Control selection with counterfactual presentation • Synthetic ground-truth validation of the method • Reproducibility testing across runs • Method-change gates in CI

21 RETAIL · AI-POWERED AUTOMATION & WORKFLOWS

Returns Adjudication Workflow

Business problem	Returns are adjudicated against policy by staff reading free-text reasons, photographs and order history under time pressure. Inconsistency is the visible problem; the invisible one is that a policy change takes weeks to show up in actual decisions.
What to build	Build a workflow that assesses synthetic return cases against policy rules held as configuration, drafts a decision citing the rule applied, and routes value- or confidence-threshold cases to a human. Handle multimodal input — free text plus supplied images — and redact personal data at the boundary before anything reaches a model, proving redaction recall on a labelled set. Prove that a policy configuration change propagates immediately by changing it and re-running the evaluation set.
Success metrics	Decision agreement with a labelled adjudication set, rule-citation accuracy, redaction recall, policy-change propagation demonstrated, human-routing precision.
Key techniques	• Multimodal input handling in a transactional workflow • Policy as version-controlled configuration • Boundary redaction with measured recall • Threshold-based routing to human review • Regression evaluation on policy change

06 CONSUMER GOODS

6 capstone briefs. Briefs across trade, demand and revenue decisions, where the hard part is reconciling sources that never agree and being honest about what did not reconcile. Each is scoped for a team of three to four from Day 3 to the panel, on synthetic data, and states the business problem, what to build, and the success metrics it will be judged on.

22 CONSUMER GOODS · MARKETING & SALES ANALYTICS

Trade Promotion ROI Workbench

Business problem	Trade spend is committed against plans whose returns are reconciled months later, if at all, across spend, shipment and retailer data that never line up on the same key. By the time the numbers are assembled the next cycle has been committed on the same assumptions.
What to build	Build the reconciliation and return calculation as a service — joining synthetic spend, shipment and retailer data and handling key mismatches explicitly rather than dropping unmatched rows silently — and surface it in a planning interface. Report unmatched volume as a first-class number, not a footnote. Write the assumptions into the repository as documented configuration. Put a data-quality gate before calculation so an incomplete period cannot produce a confident return figure.
Success metrics	Reconciliation match rate with declared unmatched volume, reproducibility across runs, gate behaviour on an incomplete period, calculation time for a full cycle.
Key techniques	• Multi-source reconciliation with explicit unmatched handling • Assumptions as documented, version-controlled configuration • Data-quality gates ahead of calculation • Planning interface with uncertainty surfaced • Reproducibility testing

23 CONSUMER GOODS · SUPPLY CHAIN & OPERATIONS ANALYTICS

Demand Signal Feature Service

Business problem	Forecasts run on shipment history and are blind to the qualitative signal in account notes, service contacts and supplier commentary — the early warning that something is slipping. That signal arrives as text, is never structured, and by the time it appears in shipment data the forecast has been wrong for a cycle.
What to build	Build a feature service combining tabular history with fields extracted from synthetic notes and commentary under a fixed schema, served with lineage to source and a stated freshness target. Feed a forecast model and quantify the lift against a tabular-only baseline. Add drift detection over the extracted features so a change in how people write is caught before it degrades the forecast. Serve the features through an API the forecast pipeline consumes, not a file someone copies.
Success metrics	Forecast error against a tabular-only baseline, feature freshness against target, drift-detection lead time, extraction schema-conformance rate.
Key techniques	• Schema-constrained extraction from unstructured text • Feature service with lineage and freshness targets • Baseline-versus-augmented lift measurement • Feature-drift detection • API-served features consumed by a pipeline

24 CONSUMER GOODS · PRICING & REVENUE OPTIMIZATION

Revenue Growth Scenario Simulator

Business problem	Price, pack and promotion decisions are debated with static spreadsheets that cannot answer the question actually being asked — what happens to volume and margin if we move this and hold that. Each scenario takes days to build, so few are tested and the analysis arrives after the decision.
What to build	Build a simulator that computes volume and margin outcomes across price, pack and promotion scenarios on synthetic elasticity data, with model assumptions stated in the interface and in code. Make scenario comparison the primary interaction, including the scenarios that lose. Bound the extrapolation explicitly — refuse to project outside the range the data supports rather than drawing a confident line through empty space. Hold a performance budget so a scenario returns fast enough to be used live in a meeting.
Success metrics	Scenario computation time at stated scenario counts, out-of-range refusal correctness, reproducibility of a scenario result, assumption-documentation coverage.
Key techniques	· Scenario engine with explicit assumption surfacing · Extrapolation bounds and refusal outside supported range · Comparison-first interface design · Performance budgeting for interactive use · Deterministic reproducibility testing

25 CONSUMER GOODS · CUSTOMER & DIGITAL ANALYTICS

Digital Shelf Compliance Monitor

Business problem	Product content on retailer sites drifts from what was published — wrong images, missing attributes, outdated claims — and is checked by someone opening pages by hand. By the time a problem is spotted it has been live through a promotion.
What to build	Build a monitoring service that compares synthetic retailer-page content against the published master record, classifies each difference by type and severity, and produces a fix queue with the evidence attached. Flag content the comparison cannot judge for review rather than passing it. Build the console with real state handling and run visual-regression checks as a gate. Instrument runs so a failed source degrades that source only, not the whole job.
Success metrics	Difference-detection precision and recall against a labelled page set, severity-classification accuracy, per-source isolation under injected failure, full-run completion time.
Key techniques	· Content comparison against a master record · Severity classification with attached evidence · Per-source failure isolation in a batch run · Production console with real state handling · Visual-regression checks as gates

26 CONSUMER GOODS · GENERATIVE AI & AGENTIC AI

Consumer Care Verbatim Insight Engine

Business problem	Consumer contacts hold the earliest signal of a product or pack problem, and they are read as a monthly theme summary that flattens exactly the detail that matters. A cluster of complaints about one pack change in one region disappears into a category average.

What to build	Build retrieval over synthetic contact verbatims supporting both thematic questions and specific lookups — a graph linking product, pack, region and issue type alongside vector retrieval over the text — and route each query to the path that answers it better. Attribute every claim to the contacts behind it so a theme can be opened and read. Evaluate on a golden set containing planted emerging issues and measure whether the system surfaces them ahead of the category average, and how early.
Success metrics	Planted-issue detection rate and lead time, citation-to-verbatim accuracy, routing correctness between graph and vector paths, cost per query.
Key techniques	• GraphRAG over a product, pack and issue graph • Graph-versus-vector query routing • Claim-to-source attribution at verbatim level • Planted-signal evaluation design • Retrieval cost and latency measurement

27 CONSUMER GOODS · AI & DATA STRATEGY / CONSULTING

Data Product Contract Service

Business problem	Downstream teams build on datasets whose shape can change without warning because nothing states what the producer guarantees. Breakages are discovered by consumers, in production, and the fix is a conversation rather than a mechanism.
What to build	Build a service holding data contracts — schema, semantics, freshness, ownership — as version-controlled artefacts, validating producer output against the contract on every run and blocking publication on a breaking change unless a version bump is declared. Notify affected consumers from the declared dependency graph. Demonstrate the whole path end to end: attempt a breaking change, see it blocked, version it, see the consumers notified.
Success metrics	Breaking-change detection rate on injected changes, false-block rate, consumer-notification completeness against the dependency graph, contract coverage of registered datasets.
Key techniques	• Contracts as version-controlled artefacts • Automated validation as a publication gate • Semantic-versioning enforcement on breaking change • Dependency-graph-driven consumer notification • Injected-change testing

07 TECHNOLOGY

6 capstone briefs. Briefs for product, support and platform systems, including two that start from a codebase nobody on the team wrote. Each is scoped for a team of three to four from Day 3 to the panel, on synthetic data, and states the business problem, what to build, and the success metrics it will be judged on.

28 TECHNOLOGY · AI-POWERED AUTOMATION & WORKFLOWS

Support Ticket Deflection and Routing Service

Business problem	Support queues mix questions already answered in the documentation with genuine defects, and everything is triaged by the same first-line read. Misrouting a defect costs a day; answering a documented question by hand costs an engineer's attention for nothing.
What to build	Build a service that classifies incoming synthetic tickets, answers the documented ones with grounded citations, and routes the rest with a predicted team and priority. Set the deflection confidence threshold and tune it explicitly against the cost of a wrong deflection versus a wrong route — then defend the trade-off you chose. Put classification and answer-quality evaluation in CI as a gate. Trace every ticket decision end to end, from input to the answer or route it produced.
Success metrics	Routing accuracy on a labelled set, deflection precision at the chosen threshold, wrong-deflection rate, p95 decision latency, gate-blocked regression demonstration.
Key techniques	• Classification with explicit confidence thresholding • Grounded answering with citation to documentation • Precision and recall trade-off tuning with a defended choice • End-to-end decision tracing • CI evaluation gates on classification and answer quality

29 TECHNOLOGY · GENERATIVE AI & AGENTIC AI

Codebase Onboarding Assistant

Business problem	New engineers spend their first weeks reconstructing how a system works from code, stale wikis and interruptions to senior engineers. The knowledge already exists in the repository and its history; nobody has made it answerable, and the wiki that tried is out of date.
What to build	Build an assistant over a supplied unfamiliar repository that indexes a code graph alongside decision history, and answers structural questions — what calls this, what breaks if I change it, why was it done this way — with citations to files, symbols and commits. Ground library questions in current documentation so the answer does not describe an API that no longer exists. Evaluate on a question set with verified answers and measure, specifically, where the assistant is confidently wrong.
Success metrics	Answer accuracy on the verified question set, citation correctness to file and symbol, confidently-wrong rate, refusal correctness on unanswerable questions, index build time.
Key techniques	• Code-graph indexing and navigation • Decision-history retrieval alongside code • Documentation grounding for library questions • Verified question-set evaluation • Explicit measurement of confident errors

30 TECHNOLOGY · AI/ML SOLUTIONS & APPLICATIONS

Product Usage Early Warning Service

Business problem	Account teams learn about churn risk when a renewal conversation goes badly. The behavioural signal — usage decay, feature abandonment, support friction — is present months earlier in product telemetry nobody has turned into a served, actionable score.
What to build	Build the feature pipeline and serve a risk score per account behind an API, returning the contributing signals alongside the score so an account team can act on it rather than just read it. Add a policy layer mapping score bands to plays, and capture what was done and what happened. Monitor score distribution and input drift in production. Build CI with an evaluation gate on a held-out period and a tested rollback, and demonstrate both.
Success metrics	Risk-ranking quality on a held-out period, signal-attribution completeness, drift-alert accuracy, p95 latency, rollback success.
Key techniques	• Telemetry feature pipeline with lineage • Scoring service returning contributing signals • Outcome capture wired back to evaluation • Production drift monitoring • CI evaluation gates with tested rollback

31 TECHNOLOGY · DATA ENGINEERING & DATA MODERNIZATION

Telemetry Pipeline Hardening

Business problem	Event telemetry degrades quietly — a field changes unit, a client version stops emitting, a schema gains a nullable that downstream assumed present. Nothing fails loudly, dashboards keep rendering, and the corruption is found when a number is questioned weeks later.
What to build	Take a supplied telemetry pipeline you did not write. Map it with a code graph, reconstruct what it actually guarantees, and write characterisation tests before changing anything. Then add quality checks as gates, drift detection over both numeric fields and text payloads, and alerting with lineage back to the emitting source. Inject degradations and report which checks catch them and how quickly. Keep the change inside a controlled blast radius and prove the pipeline still produces equivalent output.
Success metrics	Detection precision and recall on injected degradations, detection lag, output equivalence after change, characterisation-test coverage, lineage completeness.
Key techniques	• Code-graph comprehension of an unfamiliar pipeline • Characterisation testing before change • Quality checks as pipeline gates • Numeric and payload drift detection • Alert-to-source lineage

32 TECHNOLOGY · ANALYTICS & DECISION INTELLIGENCE

Cloud Unit Economics Decision Application

Business problem	Cloud spend is reported by account and service, and the question leadership asks is cost per customer, per environment and per feature. Nobody can answer it, so optimisation decisions are made against the largest line item rather than the least defensible one.

What to build	Build an application that allocates synthetic cloud cost to units of business meaning using a tagging and allocation model held as configuration, reports unallocated spend honestly as its own number, and ranks optimisation candidates by saving against stated risk. Make the allocation method inspectable in the interface rather than hidden in a query. Put a reconciliation invariant in CI — allocated plus unallocated always equals the total — and show it blocking a merge when a model change breaks it.
Success metrics	Reconciliation exactness against the total, unallocated share reported, allocation-model test coverage, blocked-merge demonstration, report generation time.
Key techniques	• Allocation model as version-controlled configuration • Honest unallocated reporting • Inspectable method surfaced in the interface • Invariant testing as a merge gate • Candidate ranking against stated risk

33 TECHNOLOGY · PRICING & REVENUE OPTIMIZATION

Packaging and Price Realisation Analyzer

Business problem	List price and realised price diverge through discounting, credits and grandfathered terms, and nobody can say where the leakage sits by segment or deal shape. Packaging changes then get proposed against list-price arithmetic the actual contracts do not support.
What to build	Build a service that computes realised price per contract from synthetic billing and contract data, decomposes the gap from list into named causes, and simulates a packaging change against the existing base rather than a hypothetical one. Where the decomposition cannot fully explain a contract, report the residual rather than distributing it silently. Expose scenario comparison in the interface and gate merges on a decomposition test against planted cases with known answers.
Success metrics	Decomposition accuracy on planted cases, residual share reported, simulation reproducibility, scenario computation time, gate demonstration.
Key techniques	• Gap decomposition into named causes • Residual reporting instead of silent allocation • Simulation against the existing contract base • Planted-case validation with known answers • Comparison-first interface design

08 MANUFACTURING

7 capstone briefs. Briefs for plant and supplier systems, where the interface runs on the floor, the workflow touches systems that fail independently, and safety limits are not advisory. Each is scoped for a team of three to four from Day 3 to the panel, on synthetic data, and states the business problem, what to build, and the success metrics it will be judged on.

34 MANUFACTURING · SUPPLY CHAIN & OPERATIONS ANALYTICS

Downtime Investigation Application

Business problem	When a line stops, causes are argued from memory in the morning meeting. Sensor traces, maintenance records, changeover logs and operator notes live in four systems, and assembling them takes longer than the outage did — so it is rarely done, and the same stop recurs.
What to build	Build an investigation application that assembles the event timeline from synthetic machine, maintenance and operator data, retrieves the relevant procedure and prior similar events with citations, and produces a candidate cause list with the evidence behind each. State plainly which candidates the data cannot distinguish between rather than picking one to look decisive. Evaluate against a labelled set of incidents with known causes and measure time-to-assembly against a manual baseline.
Success metrics	Cause-identification accuracy on labelled incidents, evidence-citation correctness, ambiguity-declaration correctness, time-to-assembly against a manual baseline.
Key techniques	• Multi-source timeline assembly • Similar-event retrieval with citation • Explicit ambiguity handling in conclusions • Labelled-incident evaluation • Investigation interface with evidence surfacing

35 MANUFACTURING · AI/ML SOLUTIONS & APPLICATIONS

Predictive Maintenance Serving Service

Business problem	A failure-prediction model has been validated and written up, and it is served nowhere maintenance planning can consume it. The distance between a working model and a serviceable one is entirely engineering, and it is the distance most teams never cross.
What to build	Package the supplied model behind an API with health probes and autoscaling, deploy it to a managed runtime, and integrate it with a mock work-order system through a typed tool interface so a prediction can become a scheduled job. Make the write idempotent so a retry cannot raise duplicate work orders. Build CI with evaluation gates, canary release and a tested rollback, then demonstrate the rollback against a deliberately bad version.
Success metrics	p95 latency, SLO adherence under the modelled load, duplicate-work-order rate under forced retries, canary rollback correctness, cold-start time.
Key techniques	• Container packaging and managed-runtime deployment • Typed tool integration to an enterprise system • Idempotent write operations • Canary release with automated rollback • CI evaluation gates before promotion

36 MANUFACTURING · GENERATIVE AI & AGENTIC AI

Maintenance Procedure Assistant

Business problem	A technician at a machine needs one step from a manual running to hundreds of pages of diagrams, tables and conditional procedures. Searching it on a plant terminal is slower than asking whoever is nearby, and the wrong procedure step on the wrong asset variant is a safety issue, not a near miss.
What to build	Build retrieval over a synthetic manual corpus where layout carries meaning — parts tables, torque specifications, conditional steps — preserving that structure through parsing and chunking. Cite the specific step and asset revision, and refuse when the asset variant is not covered rather than returning the nearest procedure. Build the interface for a plant device: fast, legible, usable with gloves. Evaluate with near-miss distractors and report precision at the top result, because the second-best procedure is a wrong answer here.
Success metrics	Precision at the top result on the distractor set, asset-variant refusal correctness, citation accuracy to step and revision, time to first token, offline-state handling.
Key techniques	• Layout-aware parsing with structure-preserving chunking • Precision-first retrieval evaluated with near-miss distractors • Variant-aware refusal rather than nearest match • Device-oriented production interface • Top-result precision measurement

37 MANUFACTURING · DATA ENGINEERING & DATA MODERNIZATION

Plant Sensor Ingestion Platform

Business problem	Sensor data arrives from lines and equipment with inconsistent tags, units and sampling rates, and is stitched together per analysis by whoever needs it. Every analysis re-derives the same cleaning slightly differently, so two teams answer the same question two ways and both are defensible.
What to build	Build the ingestion and standardisation pipeline: tag mapping, unit normalisation and resampling applied once, with quality checks as gates so a bad batch does not land. Generate the per-source mapping code from a specification with the agent and hold it to the repository standard — hooks and CI gates requiring every mapping to be documented, typed and tested before it can merge. Add drift detection on tag behaviour and lineage from any downstream value back to the raw source. Report throughput and gate behaviour under injected bad batches.
Success metrics	Mapping coverage and test completeness, gate catch rate on injected bad batches, unit-normalisation correctness against a labelled sample, lineage completeness, ingestion throughput.
Key techniques	• Specification-driven code generation under enforced standards • Hooks and CI gates requiring documentation, typing and tests • Quality checks as landing gates • Tag-behaviour drift detection • End-to-end lineage to raw source

38 MANUFACTURING · AI-POWERED AUTOMATION & WORKFLOWS

Supplier Non-Conformance Agent

Business problem	A quality non-conformance triggers a chain of steps across quality, purchasing and supplier systems — raise, notify, request corrective action, track to closure — run by email and memory. Steps get skipped, and the record of what happened is reconstructed afterwards for the audit rather than produced by the process.
What to build	Build a multi-step agent that runs the non-conformance flow against mock quality and purchasing systems through MCP tools, with checkpointing so a run resumes after failure rather than restarting. Engineer for enterprise failure conditions: idempotency on every write, circuit breakers on an unhealthy dependency, timeouts with defined fallback behaviour. Gate supplier-facing communication behind human approval. Run a chaos test under injected outage and report which guarantees held and which did not.
Success metrics	Flow-completion rate under injected outage, duplicate-write rate under forced retries, recovery-from-checkpoint success, human-approval gate correctness, audit-trail completeness.
Key techniques	• Multi-step agent with checkpointed recovery • MCP integration to enterprise systems • Idempotency keys and circuit breakers • Human approval before external communication • Chaos testing under injected outage

39 MANUFACTURING · ANALYTICS & DECISION INTELLIGENCE

Yield and Energy Trade-off Console

Business problem	Process settings are tuned for yield and for energy separately, by different people on different days, and the trade-off between them is never made explicit. Operators are asked to hold both targets without being shown where the two conflict.
What to build	Build a decision console that shows, for synthetic process data, the achievable frontier between yield and energy at the current constraints, and what a setting change costs on each axis. Hold the constraint set as explicit configuration and refuse to recommend outside the operating envelope. Record every recommendation and what the operator actually did, so recommendation quality can be scored later. Gate merges on a safety invariant: no recommendation may violate the envelope.
Success metrics	Envelope-violation rate with a target of zero, recommendation-to-action capture rate, frontier computation time, gate demonstration on an injected violating change.
Key techniques	• Constraint envelope as version-controlled configuration • Trade-off visualisation for operator decisions • Safety-invariant testing as a merge gate • Recommendation and action capture for later scoring • Performance budgeting for interactive use

40 MANUFACTURING · AI & DATA STRATEGY / CONSULTING

Agent Feasibility and Business Case for a Plant Workflow

Business problem	Not every workflow should get an agent, and the expensive version of that lesson is learned after the build. The skill being tested here is the one that saves the most money — deciding what to automate, how, and whether at all, and defending that decision to a leadership team that will ask what it costs and what it returns.
What to build	Take a plant workflow through the full analysis path: as-is and to-be process maps, decision-point modelling, and candidate automation points assessed against feasibility criteria. State plainly which candidates should be an agent, which should be simpler automation and which should stay manual — a defensible do-not-automate conclusion is a valid outcome here. Build the cost, risk and value case with bounds and sensitivity on the assumptions that actually move the answer. Then build a working agent for the single highest-feasibility candidate, to the same engineering, security and deployment standard as every other project, to prove the analysis.
Success metrics	Requirement completeness and acceptance-criteria coverage, defensibility of the feasibility decision under panel challenge, value bounds with sensitivity ranges, working agent delivered for the selected candidate.
Key techniques	• As-is and to-be process mapping with decision points • Feasibility heuristics — agent, simpler automation or manual • Cost, risk and value case with sensitivity analysis • Engineering-ready requirements handover • Working implementation of the selected candidate